

- `json.dump(request)` is used for encoding the request

```
import json
import requests
response = requests.get(url=url, params=paras)
data = json.load(response)
```

Differences while decoding with JSON

The data we get after decoding JSON encoding data is different from the original data, as shown below:

```
data = [{ 'a': 'A', 'b': (2, 4), 'c': 3.0 }]
encoded_data = json.dumps(data)
decoded_data = json.loads(encoded_data)
print 'Original data: ' data
print 'Decoded data: ' decoded_data

Output:
Original data: [{ 'a': 'A', 'c': 3.0, 'b': (2, 4)}]
Decoded data: [{ 'a': 'A', 'c': 3.0, 'b': [2, 4]}]
```

Features of Python Requests

- **Connection pooling:** There is a pool of connections, and a connection is released only once all its data has been read.
- **Sessions with cookie persistence:** You can make a session object and set certain parameters and cookie values. This allows you to persist these parameters and cookies across all requests made from the session instance.
- **Browser-style SSL verification:** We can verify the SSL certificates for HTTPS, as follows:

```
requests.get('https://example.com', verify=True)
```

For a wrong certificate, an SSL error is raised. If you don't want to verify the certificate, then:

```
requests.get('https://example.com', verify=False)
```

- **Proxies:** You can make use of proxies for sending individual requests:

```
proxy =
{
    "http": "http://10.10.1.10:3128"
}

requests.get("http://example.com", proxies=proxy)
```

- **Cookies:** We can get the cookies set by the server from the response object 'r':

```
url = 'http://example.com/cookie'
r = requests.get(url)
r.cookies['cookie_name']
```

We can also send cookies to the server, as follows:

```
url = 'http://example2.com/cookies'
cookies = dict(cookie1='This_is_a_cookie')
r = requests.get(url, cookies=cookies)
```

- **Response content:** Requests can automatically decode the response based on the header values. Using `r.encoding`, you can also change the encoding type:

```
r.encoding = 'utf-8'
```

- **Exceptions:** The various types of exceptions that are handled are:
 - Connection error: DNS failure, connection refused
 - HTTP error: Invalid HTTP response
 - Too many redirects: If the maximum number of redirects that is allowed is exceeded
- **Connection timeouts:** You can make the request stop waiting for a response after a certain time-interval. After this interval, the connection can close:

```
r = requests.get("http://example.com", timeout = 1)
```

Advantages of Python Requests

Here are the advantages of Python Requests over `urllib/urllib2`:

- Python Requests encodes the parameters automatically so you just pass them as simple arguments, unlike in

● TechnoMail
Enterprise Email Solutions

● TechnoMail Cloud
Zero Capital Email Solution

TechnoInfotech

info@technoinfotech.com
www.technoinfotech.com

● High Volume SMTP Solutions
(Hosted and on-Premise)

● Business Promotion Mailing
(Hosted and on-Premise)